What Carries Forward

A bridge from Technologies for Artificial Intelligence to Toolkit for Modern Algorithms

Fabio Antonini — Università degli Studi dell'Aquila

Not examinable · reading time about 30 minutes

Contents

What carries forward **1**
Why it is worth thirty minutes . . . 1
1. Five things that are not analogies . . . 2
1.1 Predicting the next token is lesson 4's classifier . . . 2
1.2 The softmax is lesson 9's softmax . . . 2
1.3 Retrieval is lesson 6's k-nearest neighbours . . . 2
1.4 Sampling an answer several times and voting is lesson 7's bagging . . . 3
1.5 An embedding is lesson 8's dimensionality reduction . . . 3
2. The methodology, which is the part nobody re-teaches . . . 3
2.1 Trying many prompts and reporting the best . . . 3
2.2 One split is a lottery . . . 4
2.3 Benchmark contamination is leakage . . . 4
2.4 A score without a baseline is not a result . . . 4
3. Your own numbers, pointed forward . . . 4
4. Machinery you have already built . . . 5
5. Where the connection is weaker than it looks . . . 5
What to take from it . . . 6
Where to look next . . . 6

What carries forward

This course covered no large language models, no transformers, no retrieval and no agents. That was deliberate: they belong to **Toolkit for Modern Algorithms**, which follows this one in the programme and is taught by someone else.

This document is not about them either. It is about **what you already own**, arranged by what it turns into over there.

Why it is worth thirty minutes

There is a predictable mistake waiting at the start of the next course, and it is the same shape as the ones this course kept naming.

A language model looks like a different subject. It has its own vocabulary, its own tooling, its own conferences, and demonstrations that work on the first try. The reasonable conclusion is that the ten weeks behind you were the *previous* subject, and that the new one starts from zero.

The consequence is specific and common: **you will be able to build things you cannot evaluate.** Getting a retrieval system to answer a question takes an afternoon. Knowing whether it answers better than the string-matching baseline it replaced, whether the benchmark it scores 82% on contains its own training data, and whether 82% is even different from 79% given how few questions were asked — that is this course, and nothing in the next one will teach it again because it is assumed.

Lesson 10 closed on the sentence this whole document turns on:

> A model turns a representation into a decision. **The representation usually matters more than the model.** Classical methods make you build it; convolutional networks learn it, paying in data and compute for the privilege. Neither is a default.

A language model is a representation engine of enormous generality. That changes what is easy. It changes nothing about how you decide whether a representation is any good.

---

1. Five things that are not analogies

Start here, because these are not "similar to" what you learned. They are the same objects under different names, and recognising them saves you learning anything twice.

1.1 Predicting the next token is lesson 4's classifier

Lesson 4 derived cross-entropy from maximum likelihood and made a point of saying it is **not a design choice** — it is what maximum likelihood gives you once you assume a Bernoulli outcome. That lesson's section 10 generalised it to more than two classes as **categorical cross-entropy**.

Next-token prediction is that, with the classes being the vocabulary. The loss a language model is trained on is the loss you derived in week four, over a larger set of outcomes. Nothing else about it is new.

1.2 The softmax is lesson 9's softmax

$$\text{softmax}(z)_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}$$

Lesson 9 section 5.3 wrote that down and trained ten-class networks with it. Paired with cross-entropy its gradient is again $\hat{y} - y$, the blame section 4.3 derived for one sigmoid unit; the advanced version of the lesson, in `Advanced/`, derives the ten-class case in full. It is the last layer of a language model, and the *temperature* you will meet as a sampling knob is the divisor inside the exponent.

1.3 Retrieval is lesson 6's k-nearest neighbours

Fetching relevant documents before answering is k-nearest neighbours over learned representations. The vocabulary changes — *embedding*, *vector store*, *top-k* — and the algorithm does not.

Which means **lesson 6's warnings apply without translation.** Distance needs a metric, and choosing between Euclidean and cosine is a modelling decision rather than a default. Scaling is not optional for a method made of nothing but distance. And the measurement that should worry you:

In one hundred dimensions the nearest point is **70% as far away as the farthest**.

Embeddings have 768, 1536 or 3072 dimensions. Lesson 6 measured what that does to "nearest" at a hundred. `Lessons/06_knn_naive_bayes_svm/Resources/` already carries the engineering answer — locality-sensitive hashing, product quantisation, navigable small-world graphs — which is the vector-database industry, written before you needed it.

1.4 Sampling an answer several times and voting is lesson 7's bagging

You will meet it as *self-consistency*. Lesson 7 derived why averaging reduces variance and, more usefully, why it stops:

$$\text{variance of the average} \longrightarrow \rho\sigma^2$$

The floor is set by how much the members agree with each other. Ten samples from one model at one temperature are highly correlated, so ρ is large and the floor is high. That is the same argument that told you why a random forest decorrelates its trees deliberately, and it predicts exactly where sampling more stops helping.

1.5 An embedding is lesson 8's dimensionality reduction

Lesson 8 built principal component analysis as the direction with the widest shadow and used reconstruction error to find anomalies that were invisible in any single column. An embedding is a learned, non-linear version of the same idea: a low-dimensional representation in which nearness means something.

Which carries the same consequences. Clustering embeddings to discover topics is lesson 8's k-means, with lesson 8's problem — **no labels to check against**, so silhouette tells you a clustering is self-consistent and nothing about whether it is right. Detecting out-of-distribution inputs by reconstruction error is lesson 8's section 5, unchanged.

---

2. The methodology, which is the part nobody re-teaches

If you keep one section of this document, keep this one. The identities above are pleasant; the following is what separates people who can evaluate a system from people who can only demonstrate one.

2.1 Trying many prompts and reporting the best

Lesson 5 measured it on a table with **no signal in it whatsoever** — the true area under the curve was **0.500**, by construction. A grid of 25 candidate configurations, each scored on the same split, reported a best of **0.7999**. Nested cross-validation, which refits the selection inside every fold, gave **0.6699** on the identical data.

The gap between those two numbers is what selecting on the data you report costs. It is not a small correction: one procedure says the model is good and the other says there was nothing there.

Prompt engineering is model selection. Every prompt you try is a candidate; every candidate you evaluate on the same small set of examples buys you another chance to be fooled. `Resources/the_replication_crisis.md` from lesson 5 names the general form — Gelman and Loken's **garden of forking paths** — and it is the single most transferable idea in this course.

2.2 One split is a lottery

Lesson 5's measurement: with the selection correctly inside the pipeline, the estimate over twenty fold seeds settles at **0.658** — but the individual splits range from **0.576 to 0.759**. One split can hand you either end, and it will not tell you which one it handed you.

Now add that language models are **stochastic at inference**. You have two sources of variation where this course had one, and the discipline you learned — repeat, report the spread, never rank on a single run — matters more, not less.

Lesson 7 taught the same lesson from the other side. Two scores agreeing to four decimal places looked like a law and turned out to be one seed's luck: across twelve seeds the typical gap was 0.0026. If you find yourself writing *exactly*, run it again.

2.3 Benchmark contamination is leakage

Lesson 2 built a leak that raised a score, broke no rule visibly, and threw no error — an imputer fitted before the split, so that three quarters of the affected training rows had a test row among the five neighbours it consulted. Lesson 5 gave it its name.

The same failure, at the scale of the internet, is a benchmark whose questions are in the training corpus. It is the dominant methodological problem of the field you are entering, it is *your* lesson 2 and lesson 5, and it is why a headline benchmark number deserves the question **"where did the test set come from, and could the model have seen it?"** before any other question.

2.4 A score without a baseline is not a result

Lesson 1's first measurement was a classifier that answers "negative" every time, scores 0.986, and finds nothing. The trained model beside it scored 0.987.

"The model gets 71% on our evaluation" is not a result until you know what answering C every time gets, what the previous system got, and what a competent human gets. This is the cheapest habit in the course and the one most often skipped when the system is impressive.

---

3. Your own numbers, pointed forward

From	Number	What it warns you about there
Lesson 6	nearest is **70%** as far as farthest at 100 dimensions	retrieval quality in 1536-dimensional embedding spaces

From	Number	What it warns you about there
Lesson 7	**54%** of a forest's importance landed on pure noise while accuracy degraded gracefully	reading attention weights as explanations
Lesson 9	six sigmoid layers never left chance, **0.1000**, where the same network with ReLU reached 0.9611	why residual connections and normalisation exist at all
Lesson 4	a false alarm costs €140, a miss €2,600	every guardrail, filter and router is a threshold on a cost
Lesson 8	**37 of 40** planted anomalies caught, and 32–38 across other draws	quoting one evaluation run as a specification
Lesson 10	a random forest on raw pixels came within **two thousandths** of a convolutional network	assuming the newest architecture wins

The lesson 7 entry deserves a sentence of its own. Its forest kept a respectable accuracy while more than half of the importance it reported had landed on columns that were pure noise — **the ranking was wrong in a way the accuracy gave no hint of.** Attention weights are the same kind of artefact: a number the model produces, which is not the same as a reason the model acted.

---

4. Machinery you have already built

Two of the next course's components are things you wrote yourself. Open them again and read them as what they become — this is cheaper than any tutorial.

- `Lessons/06_knn_naive_bayes_svm/Notebooks/01_knn_and_the_curse.ipynb` — you implemented nearest-neighbour search in about eight lines and then watched it degrade as dimensions were added. That is a retrieval system and its failure mode, built by you.
- `Lessons/08_unsupervised_learning/Notebooks/03_pca_and_anomaly_detection.ipynb` — you reduced eight columns to three, reconstructed, and used the error to find anomalies invisible in any single column. That is the embedding-space machinery, minus the learning.
- `Lessons/09_neural_networks/Notebooks/01_backpropagation_from_scratch.ipynb` — you wrote backpropagation and checked it against finite differences. A transformer is those layers with a different mixing operation between them.

---

5. Where the connection is weaker than it looks

An honest bridge says where it stops.

Scale changes which problems exist. This course fitted models on thousands of rows on a laptop with no graphics card. At the scale of a language model, the failures that dominate —

distributed training, memory, throughput, inference cost — are engineering problems this course never posed.

Some of what you learned about interpretability does not transfer cleanly. A decision tree's path is an explanation. Lesson 7's feature importances were already a caution; the next course's equivalents are weaker still, and lesson 7's Resources document on GDPR Article 22 and the right to an explanation is the right place to have your expectations set.

Evaluation without ground truth is genuinely harder there. Lesson 8 said that clustering has no answer key and that internal metrics measure self-consistency rather than correctness. A great deal of language-model evaluation is in that position, and the honest answer is that the field has not solved it. What lesson 8 gives you is the reflex to notice when you are in that position rather than to assume you are not.

What to take from it

You are not starting a new subject. You are meeting the same objects with different names, plus one genuinely new architectural idea.

The methodology is not assumed knowledge that you may skip — it is the part that will distinguish you. A great many people can make a language model produce something impressive. Rather fewer can say whether it is better than what it replaced, and defend the claim.

And the question this whole course kept asking is the one to carry across unchanged. Not *is this number correct* — but:

> **Does this sentence claim more than this number supports?**

Where to look next

Source	Type	Why
`Lessons/05_experimental_methodology/Resources/the_replication_crisis.md`	this course	The garden of forking paths, and why the problem is not one of beginners
`Lessons/06_knn_naive_bayes_sm/Resources/finding_neighbours_at_scale.md`	this course	Approximate nearest-neighbour search: the vector-database industry, from first principles
`Lessons/07_trees_and_ensembles/Resources/right_to_explanation.md`	this course	GDPR Article 22 and what an explanation is legally required to be
Gelman and Loken, *The Garden of Forking Paths* (2013)	paper	The original statement of the multiple-comparisons problem without explicit multiple comparisons

Source	Type	Why
Leskovec, Rajaraman and Ullman, *Mining of Massive Datasets*, chapter 3	book, free online	Similarity search at scale, derived rather than described